PeakWeights: Data-Free Discovery of Critical LLM Parameters

Thiyagarajan M*[1] and Vamshi Ambati[2]

[1]Kalmantic Labs
[2]Carnegie Mellon University; IIIT Hyderabad

December 2025

Abstract

Post-training quantization of large language models introduces errors that degrade quality, but not all weights contribute equally to this degradation. PeakWeights identifies critical weights in a single forward pass without calibration data. Weight magnitude multiplied by peak activation closely tracks worst-case quantization error. Across five architectures (SmolLM2-1.7B, Qwen2.5-7B, DeepSeek-R1-7B, Mistral-7B, Phi-3-mini), protecting 50 weights during 4-bit quantization recovers 61-99% of lost perplexity. The optimal number of protected weights is architecture-dependent: four models achieve 90%+ recovery at K=50, while Mistral requires K=100. Weight importance follows a power law ($\alpha \approx 0.39$-0.49), but critical weight locations vary: MLP layers for Qwen/DeepSeek, output projection for Mistral. Code available at https://github.com/Kalmantic/peakweights.

1 Introduction

Quantization reduces the memory footprint of large language models by representing weights in lower precision [Dettmers et al., 2022, Frantar et al., 2022]. The cost is quality degradation: 4-bit quantization increases perplexity by 0.36 to 6.64 points across models we tested.

Recent work has shown that this degradation is not uniform. A small number of weights, variously called "outliers" or "super weights," have disproportionate influence on model outputs [Yu et al., 2024]. Protecting these weights during quantization recovers quality. The open question is how to find them efficiently.

Existing methods require calibration data and multiple forward passes, but a single forward pass with synthetic input suffices. Quantization error at weight w_{ij} propagates to the output proportionally to the activation x_j, so weights that are both large and see large activations cause the most damage when quantized.

2 Related Work

Post-Training Quantization. GPTQ [Frantar et al., 2022] minimizes layer-wise quantization error using approximate Hessian information, requiring calibration data. AWQ [Lin et al., 2024] identifies salient channels through activation magnitudes and applies per-channel scaling. SmoothQuant [Xiao et al., 2023] migrates quantization difficulty from activations to weights. These methods achieve strong results but require representative data samples.

Outlier Weights. Dettmers et al. [2022] observed that a small fraction of hidden dimensions contain activation outliers that cause quantization failures. Concurrent work by Yu et al. [2024] identifies individual "super weights" whose removal catastrophically degrades performance. We extend this analysis by characterizing architecture-dependent optimal K values rather than focusing on a single weight, and by providing power-law analysis ($\alpha \approx 0.40$-0.46) that explains why different architectures require different protection strategies.

*thiyagarajan@kalmantic.com

3 Method

3.1 Scoring Function

Consider a linear layer $\mathbf{y} = \mathbf{W}\mathbf{x}$. Quantizing weight w_{ij} introduces error δ_{ij}, perturbing the output by $\delta_{ij} \cdot x_j$. For uniform quantization, larger weights incur larger errors: $|\delta_{ij}| \propto |w_{ij}|$.

Each weight is scored by worst-case output perturbation:

$$\text{score}(w_{ij}) = |w_{ij}| \times \max_t |x_{t,j}| \tag{1}$$

where $x_{t,j}$ is the activation at position j for token t. The maximum captures worst-case behavior; even rare large activations matter.

3.2 Data-Free Activation Estimation

Activation magnitudes are estimated using synthetic input: the tokenized string "0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15" (16 tokens). The goal is identifying channels that *can* produce large activations, not channels that typically do. Network structure dominates input choice.

3.3 Algorithm

Algorithm 1 PeakWeights

```
Register forward hooks on Linear, Embedding, Conv1d modules
Forward pass with synthetic tokens
for each module do
    Record max_t |x_{t,j}| for each input feature j
end for
for each weight w_ij do
    score_ij ← |w_ij| × max_t |x_{t,j}|
    Update min-heap of top-K scores
end for
return Top-K weights
```

Runtime is $O(P)$ with $O(K)$ space. A 7B model completes in approximately 50 seconds on an A100 80GB.

4 Experiments

4.1 Setup

We evaluate on five models with different architectures:

- **SmolLM2-1.7B**: Standard transformer, 1.7B parameters
- **Qwen2.5-7B**: SwiGLU MLP, grouped-query attention
- **DeepSeek-R1-Distill-Qwen-7B**: Reasoning-focused, distilled
- **Mistral-7B-v0.3**: Sliding window attention, tied embeddings
- **Phi-3-mini**: Microsoft's efficient 3.8B model

Perplexity is measured on WikiText-103 [Merity et al., 2016]. We simulate 4-bit quantization and define recovery as:

$$\text{Recovery} = \frac{\text{PPL}_{\text{4bit}} - \text{PPL}_{\text{protected}}}{\text{PPL}_{\text{4bit}} - \text{PPL}_{\text{FP16}}} \tag{2}$$

4.2 Main Results

Table 1: Perplexity on WikiText-103. Lower is better.

Model	FP16	4-bit	PeakWeights	Recovery
Qwen2.5-7B	10.62	11.51	10.63	99%
Mistral-7B	13.44	13.80	13.58	61%
SmolLM2-1.7B	17.92	24.56	17.99	99%
DeepSeek-R1-7B	70.73	73.52	70.91	93%
Phi-3-mini	11.65	12.67	11.68	97%

Four models exceed 90% recovery at K=50. Mistral requires more protected weights due to a flatter importance distribution (α=0.44 vs 0.45-0.49 for others).

4.3 Effect of K

Table 2: Recovery rate (%) at different K. Bold indicates first K achieving $\geq$90%.

Model	K=1	K=5	K=10	K=20	K=50	K=100
Qwen2.5-7B	18	47	64	78	**99**	99
Mistral-7B	2	7	13	25	61	**99**
SmolLM2-1.7B	39	69	80	89	**99**	99
DeepSeek-R1-7B	12	35	49	68	**93**	99
Phi-3-mini	8	31	51	71	**97**	99

The optimal K varies by a factor of 5$\times$ across models. Fixed "core-K" assumptions do not hold.

4.4 Score Distribution

Weight scores follow a power law: score $\propto$ rank$^{-\alpha}$.

Table 3: Power law exponents. Higher α indicates concentrated importance.

Model	Exponent (α)
Qwen2.5-7B	0.46
Mistral-7B	0.44
SmolLM2-1.7B	0.49
DeepSeek-R1-7B	0.39
Phi-3-mini	0.45

SmolLM2's steeper exponent (0.49) means importance is concentrated in fewer weights. DeepSeek-R1's flatter distribution (0.39) and Mistral's moderate exponent (0.44) explain why they require more protected weights.

4.5 Critical Weight Location

Table 4: Distribution of top-10 critical weights by module type.

Model	MLP	Attention	lm_head
Qwen2.5-7B	100%	0%	0%
Mistral-7B	10%	0%	90%
SmolLM2-1.7B	100%	0%	0%
DeepSeek-R1-7B	100%	0%	0%
Phi-3-mini	100%	0%	0%

Location reflects architecture. Qwen and DeepSeek use SwiGLU MLPs, creating bottlenecks in `down_proj`. Mistral uses tied embeddings, concentrating importance in the output projection.

5 Analysis

Model Size and Redundancy. Smaller models show higher recovery at fixed K. SmolLM2 (1.7B) achieves 99% at K=50; Mistral (7B) achieves 61%. Larger models have more redundant pathways, diluting the impact of protecting individual weights.

Reasoning Models. DeepSeek-R1 shows 70.73 perplexity on WikiText-103, significantly higher than language-modeling-focused models. This reflects training objective mismatch, not model quality. PeakWeights still achieves 93% recovery, indicating the method generalizes across training objectives.

Automatic K Selection. Given target recovery r, we compute cumulative importance $C(k) = \sum_{i=1}^{k} \text{score}_i / \sum_i \text{score}_i$ and return the smallest k where $C(k) \geq r$.

6 Limitations

Synthetic input may not capture activation patterns arising only from specific real inputs. Results are validated on models up to 7B parameters; larger models may exhibit different behavior. This work makes no claim about interpretability or semantic localization; it is an empirical result about functional redundancy under inference-time metrics.

7 Conclusion

PeakWeights identifies critical weights through a simple scoring function: weight magnitude times maximum activation. One forward pass with synthetic input suffices; no calibration data required.

The number of weights requiring protection is architecture-dependent. Four models achieve 90%+ recovery at K=50; Mistral needs K=100. Methods assuming a fixed count will underperform on some architectures.

We see power-law distributions in all five models, though the exponents differ (α=0.39-0.49), which explains why different architectures need different K. Critical weights cluster in different modules depending on architecture: MLP for most models, output layer for tied-embedding models like Mistral.

Code and Reproducibility

The PeakWeights algorithm is implemented as an open-source Python package at https://github.com/Kalmantic/peakweights. Install with `pip install peakweights`.

Usage. Find critical weights for any HuggingFace model:

```
from peakweights import find
critical = find("Qwen/Qwen2.5-7B", k=50)
```

Quantization Integration. Protect critical weights during 4-bit quantization:

```
from transformers import BitsAndBytesConfig
modules = list(set(w.module for w in critical[:10]))
config = BitsAndBytesConfig(
    load_in_4bit=True,
    llm_int8_skip_modules=modules
)
```

Reproducing Results. The repository includes a Google Colab notebook with all experiments from this paper. To reproduce: open the notebook, select A100 runtime (free tier), and run all cells. Takes under 30 minutes.

References

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *NeurIPS*, 2022.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *arXiv preprint arXiv:2210.17323*, 2022.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. *MLSys*, 2024.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer Sentinel Mixture Models. *arXiv preprint arXiv:1609.07843*, 2016.

Mengxia Yu, De Wang, Qi Shan, Colorado Reed, and Alvin Wan. The Super Weight in Large Language Models. *arXiv preprint arXiv:2411.07191*, 2024.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. *ICML*, 2023.